

強化學習中的虛實轉移差距 (Sim-to-Real gap)：以 TIAGo 與 Isaac Sim/Gym 為例

Jaume Albardaner, Alberto San Miguel, Néstor García, and
Magí Dalmau

Eurecat, Centre Tecnològic Catalunya, Robotics &
Automation Unit, Barcelona, Spain

{jaume.albardaner, alberto.sanmiguel, nestor.garcia,
magi.dalmau}@eurecat.org

摘要

本論文探討了在使用 TIAGo 行動操作機器人進行機器人操作的虛實轉移 (sim-to-real transfer) 背景下的策略學習方法，並聚焦於由 Nvidia 開發的兩款尖端模擬器：Isaac Gym 與 Isaac Sim。文中討論了控制架構，特別強調在模擬與真實環境中實現無碰撞運動。研究結果展示了成功的虛實轉移，呈現了經由強化學習 (RL) 訓練的模型在模擬與真實設置中執行相似運動的成果。

Keywords: Reinforcement Learning, Isaac Gym, Isaac Sim, TIAGo, Sim2Real.

1 Reinforcement Learning simulators

Reinforcement Learning (RL) techniques are especially useful for applications that require a certain degree of autonomy. In Robotics, this applies to tasks related to object manipulation and navigation, and the main difficulty lies on the gathering of data to train a model. An economically expensive approach to make it faster is to acquire multiple robots and run them simultaneously [1, 2, 3]. Models trained with that data are more likely to work on the real robot, since the data was obtained from the same platform the model runs on. However, performing RL on real robots still requires supervision to ensure that no unexpected scenario is met. This approach is not only expensive, it also requires a lot of time for data generation. Not only because of how slow the robot may move, but also because the environment may need to be prepared before every run.

為了滿足讓過程更快速且成本更低的開發需求，業界開發了多個強化學習 (RL) 函式庫 (例如 OpenAI 的 Gymnasium 4 或 Stable-baselines 5)，並將機器人模型引入模擬器中 (如 Mujoco [6])。雖然這個過程確實降低了成本，但並不一定能加快速度。隨著硬體發展，支援 GPU 的模擬器相繼出現，其中 Nvidia 的 IsaacGym [7] 與 IsaacSim [8] 表現尤為出色。儘管如此，模擬器內發生的情況終究只是現實的簡化。因此，若要將這些模型應用於真實環境，必須進行適配。這是 RL 中一個非常活躍的分支，稱為模擬轉真實 (sim-to-real 或 Sim2Real)，也是本文的主要研究焦點。

2 TIAGo 使用案例

2.1 模型

為不具備簡單網格 (meshes) 的物體進行物理模擬，在計算上非常昂貴。如果可能的話，模擬器會將這些網格簡化為凸形 (convex shapes) 以加快模擬速度。此外，在大多數模擬器中，機器人的預設設定通常不包含自我碰撞檢查。

TIAGo 行動操作機器人¹的情況正是如此，必須更改模擬器設定以允許自我碰撞 (self-collision)，且由於網格簡化 (mesh simplification) 導致的自我碰撞，機器人模型也必須進行修改。

2.2 控制 (Controls)

機器人的控制方式以及對控制輸入的反應在每個模擬器中都有所不同。本節將說明所使用的模擬器以及真實機器人的控制流程 (control pipeline)。

Isaac Gym 可應用三種類型的關節控制：位置 (position)、速度 (velocity) 和力矩 (effort) 控制。如圖 1 所示，當位置或速度的參考值更新時，會使用 PD 控制器來追蹤新的參考值。PD 控制器從每個關節的剛度 (stiffness) 和阻尼 (damping) 參數中取得其 k_p 與 k_d 值。產生的控制指令會傳送至機器人，確保不違反任何關節限制，並模擬其對每個關節的影響。

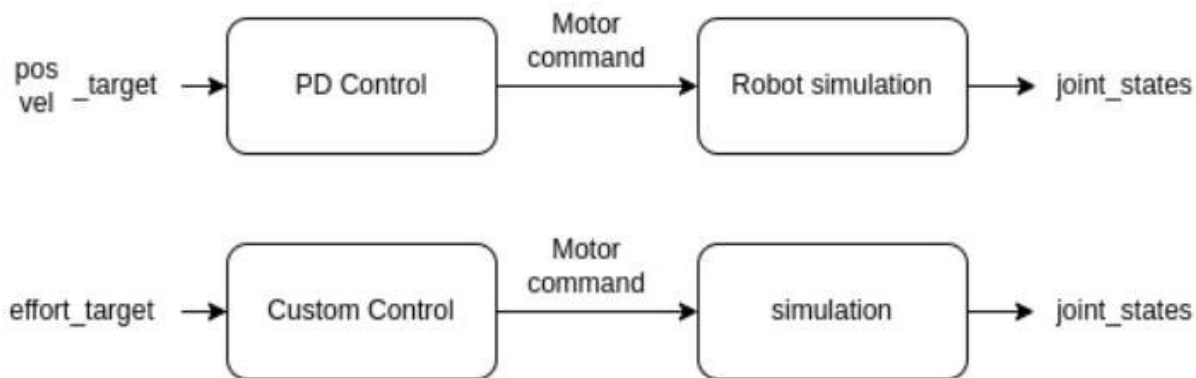


圖 1: Isaac Gym 與 Isaac Sim 的控制流程圖。

在 Isaac Gym 中，並未內建針對力矩控制（effort control）的控制器。然而，Nvidia 的 IsaacGymEnvs GitHub 儲存庫²中提供了範例控制器。

Isaac Sim 由於此模擬器同樣依賴 PhysX 且由 Nvidia 開發，其遵循的控制流程與圖 1 相同。唯一的區別在於它提供了多種現成的控制器。這些控制器可以在關節空間（joint space）和笛卡兒空間（cartesian space）中操作機器人。

TIAGo 本案例中所使用的機器人遵循由 Robot Operating System (ROS) 套件 `ros_control`³ 所建立的控制架構（圖 2）。由於該套件是由社群驅動的，因此流程中的每個步驟都可以獨立進行自定義與評估。

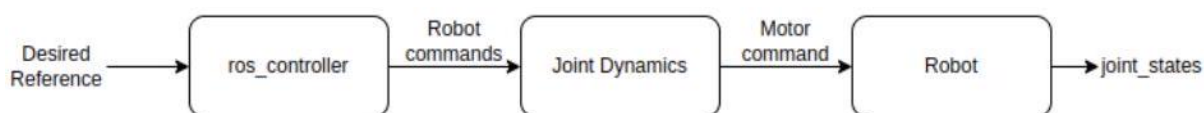


圖 2: 實體 TIAGo 的控制流程圖。

以 TIAGo 為例，我們採用了 `JointGroupPosition` 控制器。選擇此控制器是因為它與 Isaac 的 PD 最為相似：它是一個 PID 控制器。

3 結果

3.1 模擬器回應

為了評估各個模擬器與真實機器人之間的響應差異，我們對手臂的每個關節引入了階躍輸入（step input）。雖然在兩個模擬器中，單個關節的響應非常相似，但當多個關節同時引入階躍輸入時，其響應則會出現差異。

圖 3 顯示，Isaac Gym 的手臂響應比 Isaac Sim 更接近真實 TIAGo 的結果。儘管如此，它的響應似乎較慢，這可能是由於根據關節在階層樹（hierarchical tree）中的位置來決定優先順序所致。此假設是基於累積誤差的演變，其中 DOF 7 的誤差收斂速度似乎比 DOF 1 慢。此外，如表 1 所示，Isaac Gym 的累積誤差也比 Isaac Sim 小。另一方面，與 Isaac Sim 的響應差異很可能是由於未遵守關節層級的速度限制，如圖 4 所示。

	DOF 1	DOF 2	自由度 3	自由度 4	自由度 5	自由度 6	7 自由度 (DOF)	Σ
Isaac Gym	8.4202	7.5179	26.4569	9.0018	43.6909	6.46288	12.7499	114.3006
Isaac Sim	6.6786	4.6277	16.5728	9.1713	11.9117	46.1073	57.0088	152.0785

表 1: 達到穩態後各自由度 (DOF) 的累積誤差 (rad)。最小值以粗體標示。

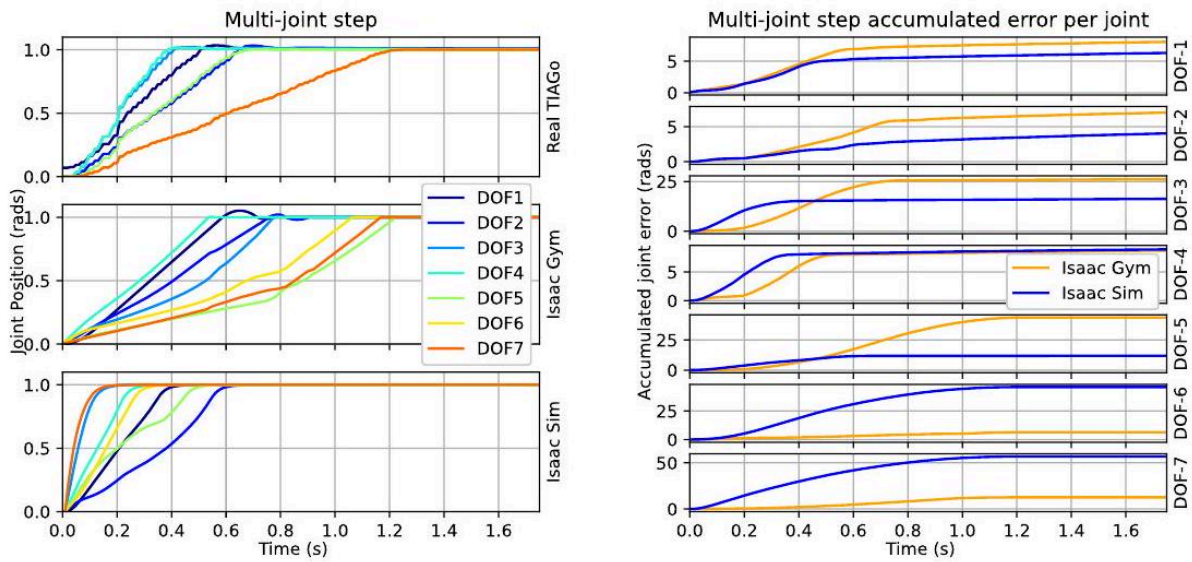


圖 3: 圖 3: 真實 TIAGo、Isaac Gym 與 Isaac Sim 中，TIAGo 手臂各個自由度 (DOF) 對各關節同時進行階躍輸入 (step inputs) 的響應 (a)，以及各模擬器相對於真實機器的累積誤差 (b)

Joint velocities in each simulator

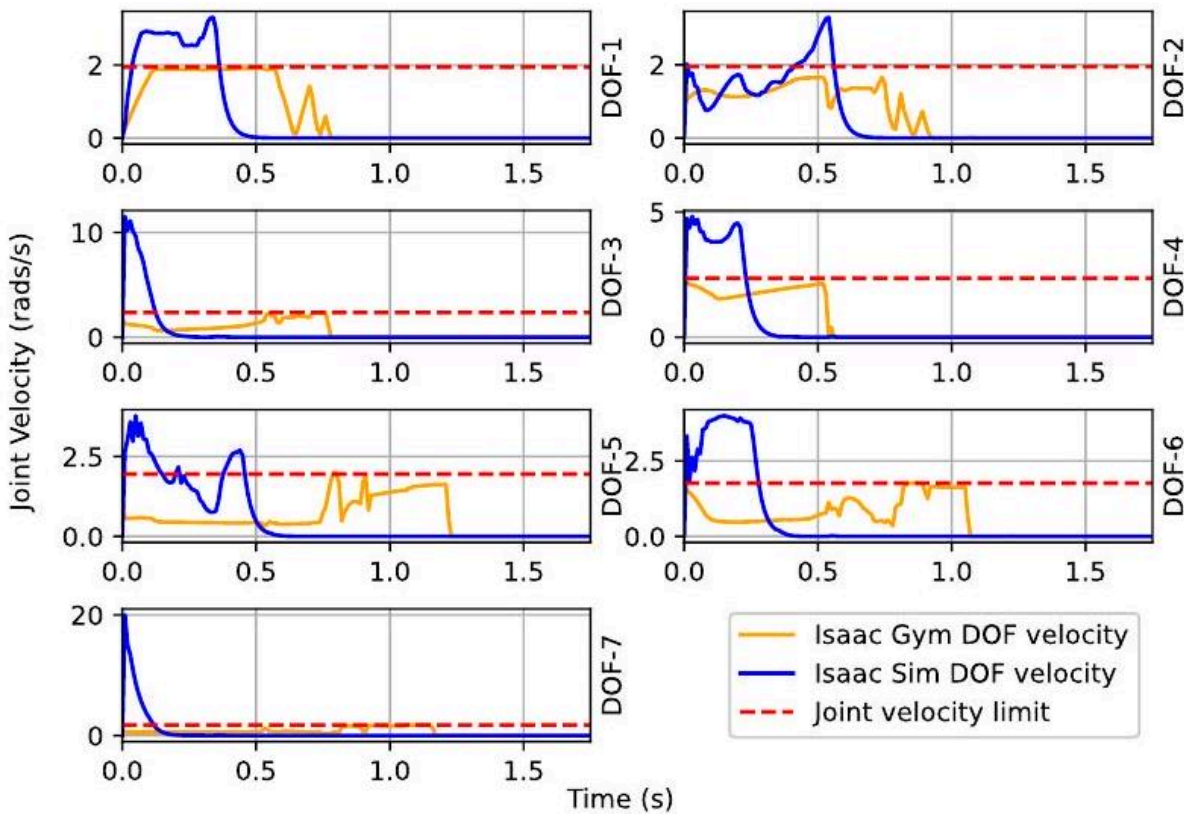


圖 4: 圖 4: Isaac Gym 與 Isaac Sim 中，TIAGo 手臂各個自由度對各關節同時進行階躍輸入的運作速度

3.2 已訓練模型

使用此設置訓練的第一個模型，是將 TIAGo 行動操作器從其「Home」位置移動到手臂完全伸展的位置（圖 5），這對應於所有關節皆處於零位（zero position）。

在圖 5 中可以看到，儘管兩個模型都使用相同的獎勵函數（ $\text{reward} = -|DOF_{\text{value}}|$ ）並經過相同數量的訓練週期（training epochs (100 K)）進行訓練，但其動作表現仍有所不同。

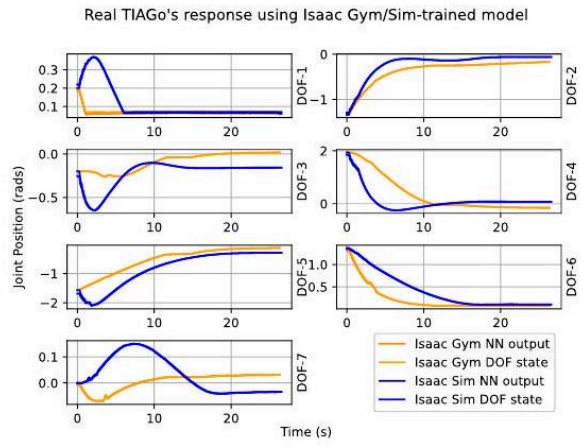
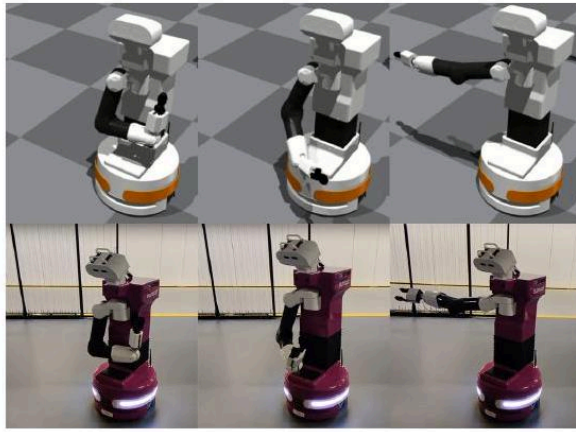


圖 5: TIAGo 在 Isaac Gym 中的 Home-to-zero 動作 (左上)、在真實環境中的動作 (左下)，以及在 Isaac Gym 與 Isaac Sim 中訓練的模型之 DOF 位置隨時間變化的情形 (右)。

4 結論

本文將策略學習方法應用於機器人平台的 Sim-to-Real 轉移，特別是針對 TIAGo 移動操作機器人使用 Isaac Gym 與 Isaac Sim 模擬器。雖然呈現的結果展示了令人期待的成果，但未來仍需解決已識別的問題，以縮小此案例中的 Sim-to-Real 差距，並評估其對其他機器人平台的影響。

參考文獻

- Levine S , Pastor P, Krizhevsky A, Ibarz J, Quillen D. Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. The International Journal of Robotics Research. 2018;37(4-5):421-436. doi: 10.1177/0278364917710318
- Gu, Shixiang, et al. "Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates." 2017 IEEE international conference on robotics and automation (ICRA). IEEE, 2017.
- Kalashnikov, Dmitry, et al. "Scalable deep reinforcement learning for vision-based robotic manipulation." Conference on Robot Learning. PMLR, 2018.
- M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. D. Cola, T. Deleu, et al., Gymnasium, Mar. 2023, [online] Available: <https://zenodo.org/record/8127025>.
- Raffin, Antonin, et al. "Stable-baselines3: Reliable reinforcement learning implementations." The Journal of Machine Learning Research 22.1 (2021): 12348-12355.
- Todorov, Emanuel, Tom Erez, and Yuval Tassa. "Mujoco: A physics engine for model-based control." 2012 IEEE/RSJ international conference on intelligent robots and systems. IEEE, 2012.
- Makoviychuk, Viktor, et al. "Isaac gym: High performance gpu-based physics simulation for robot learning." arXiv preprint arXiv:2108.10470 (2021).
- NVIDIA, Isaac Sim 機器人模擬器, <https://developer.nvidia.com/isaac-sim> (造訪於 2024 年 3 月 28 日)。

¹ pal-robotics.com/robots/TiaGo/
² github.com/NVIDIA-Omniverse/IsaacGymEnvs
³ github.com/ros-controls/ros_control